

Exercise 3.1: Install Kubernetes

Overview

There are several Kubernetes installation tools provided by various vendors. In this lab we will learn to use **kubeadm**. As a community-supported independent tool, it is planned to become the primary manner to build a Kubernetes cluster.



Platforms: Digital Ocean, GCP, AWS, VirtualBox, etc

The labs were written using **Ubuntu 20.04** instances running on **Google Cloud Platform (GCP)**. They have been written to be vendor-agnostic so could run on AWS, local hardware, or inside of virtualization to give you the most flexibility and options. Each platform will have different access methods and considerations. As of v1.21.0 the minimum (as in barely works) size for **VirtualBox** is 3vCPU/4G memory/5G minimal OS for cp and 1vCPU/2G memory/5G minimal OS for worker node. Most other providers work with 2CPU/7.5G.

If using your own equipment you will have to disable swap on every node, and ensure there is only one network interface. Multiple interfaces are supported but require extra configuration. There may be other requirements which will be shown as warnings or errors when using the **kubeadm** command. While most commands are run as a regular user, there are some which require root privilege. You If you are accessing the nodes remotely, such as with **GCP** or **AWS**, you will need to use an SSH client such as a local terminal or **PuTTY** if not using **Linux** or a Mac. You can download **PuTTY** from www.putty.org. You would also require a **.pem** or **.ppk** file to access the nodes. Each cloud provider will have a process to download or create this file. If attending in-person instructor led training the file will be made available during class.



Very Important

Please disable any firewalls while learning Kubernetes. While there is a list of required ports for communication between components, the list may not be as complete as necessary. If using **GCP** you can add a rule to the project which allows all traffic to all ports. Should you be using **VirtualBox** be aware that inter-VM networking will need to be set to promiscuous mode.

In the following exercise we will install Kubernetes on a single node then grow the cluster, adding more compute resources. Both nodes used are the same size, providing 2 vCPUs and 7.5G of memory. Smaller nodes could be used, but would run slower, and may have strange errors.



YAML files and White Space

Various exercises will use YAML files, which are included in the text. You are encouraged to write some of the files as time permits, as the syntax of YAML has white space indentation requirements that are important to learn. An important note, **do not** use tabs in your YAML files, **white space only. Indentation matters.**

If using a PDF the use of copy and paste often does not paste the single quote correctly. It pastes as a back-quote instead. You will need to modify it by hand. The files mentioned in labs have also been made available as a compressed **tar** file. You can view the resources by navigating to this URL:

<https://cm.lf.training/LFS258>

To login use user: LFtraining and a password of: Penguin2014

Once you find the name and link of the current file, which will change as the course updates, use **wget** to download the file into your node from the command line then expand it like this:

```
$ wget https://cm.lf.training/LFS258/LFS258.V2024-02-22_SOLUTIONS.tar.xz \
    --user=LFtraining --password=Penguin2014

$ tar -xvf LFS258.V2024-02-22_SOLUTIONS.tar.xz
```

(**Note:** depending on your PDF viewer, if you are cutting and pasting the above instructions, the underscores may disappear and be replaced by spaces, so you may have to edit the command line by hand!)

Install Kubernetes

Log into your control plane (cp) and worker nodes. If attending in-person instructor led training the node IP addresses will be provided by the instructor. You will need to use a **.pem** or **.ppk** key for access, depending on if you are using **ssh** from a terminal or **PuTTY**. The instructor will provide this to you.

1. Open a terminal session on your first node. For example, connect via **PuTTY** or **SSH** session to the first **GCP** node. The user name may be different than the one shown, **student**. Create a non-root user if one is not present. The IP used in the example will be different than the one you will use. You may need to adjust the access mode of your pem or ppk key. The example shows how a Mac or Linux system would change mode. Windows may have a similar process.

```
[student@laptop ~]$ chmod 400 LFS258.pem
[student@laptop ~]$ ssh -i LFS258.pem student@35.226.100.87
```

```
The authenticity of host '54.214.214.156 (35.226.100.87)' can't be established.
ECDSA key fingerprint is SHA256:IPvznbkx93/Wc+ACwXrCcDDgvBwmvEXC9vmYhk2Wo1E.
ECDSA key fingerprint is MD5:d8:c9:4b:b0:b0:82:d3:95:08:08:4a:74:1b:f6:e1:9f.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '35.226.100.87' (ECDSA) to the list of known hosts.
<output_omitted>
```

2. Use the **wget** command above to download and extract the course tarball to your node. Again copy and paste won't always paste the underscore characters.
3. You are encouraged to type out commands, if using a PDF or eLearning, instead of copy and paste. By typing the commands you have a better chance to remember both the command and the concept. There are a few exceptions, such as when a long hash or output is much easier to copy over, and does not offer a learning opportunity.
4. Become **root** and update and upgrade the system. You may be asked a few questions. If so, allow restarts and keep the local version currently installed. Which would be a yes then a 2.

```
student@cp:~$ sudo -i
```

```
root@cp:~# apt-get update && apt-get upgrade -y
```

```
<output_omitted>
```

```
You can choose this option to avoid being prompted; instead,
all necessary restarts will be done for you automatically
so you can avoid being asked questions on each library upgrade.
```

```
Restart services during package upgrades without asking? [yes/no] yes
```

```
<output_omitted>
```

```
A new version (/tmp/fileEbke6q) of configuration file /etc/ssh/sshd_config is
available, but the version installed currently has been locally modified.
```

1. install the package maintainer's version
2. keep the local version currently installed
3. show the differences between the versions
4. show a side-by-side difference between the versions
5. show a 3-way difference between available versions
6. do a 3-way merge between available versions
7. start a new shell to examine the situation

What do you want to do about modified configuration file sshd_config? 2

<output_omitted>

5. Install a text editor like **nano** (an easy to use editor), **vim**, or **emacs**. Any will do, the labs use a popular option, **vim**.

```
root@cp:~# apt-get install -y vim
```

<output-omitted>

6. The main choices for a container environment are **containerd**, **cri-o**, and **Docker** on older clusters. We suggest **containerd** for class, as it is easy to deploy and commonly used by cloud providers.

Please note, install one engine only. If more than one are installed the **kubeadm** init process search pattern will use Docker at the moment. Also be aware that engines other than **containerd** may show different output on some commands.

7. There are several packages we should install to ensure we have all dependencies take care of. Please note the backslash is not necessary and can be removed if typing on a single line.

```
root@cp:~# apt install curl apt-transport-https vim git wget \
software-properties-common lsb-release ca-certificates -y
```

<output-omitted>

8. Disable swap if not already done. Cloud providers disable swap on their images.

```
root@cp:~# swapoff -a
```

9. Load modules to ensure they are available for following steps.

```
root@cp:~# modprobe overlay
root@cp:~# modprobe br_netfilter
```

10. Update kernel networking to allow necessary traffic. Be aware the shell will add a greater than sign (>) to indicate the command continues after a carriage return.

```
root@cp:~# cat << EOF | tee /etc/sysctl.d/kubernetes.conf
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
EOF
```

```
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
```

11. Ensure the changes are used by the current kernel as well

```
root@cp:~# sysctl --system
```

```
* Applying /etc/sysctl.d/10-console-messages.conf ...
kernel.printk = 4 4 1 7
```

```
* Applying /etc/sysctl.d/10-ipv6-privacy.conf ...
net.ipv6.conf.all.use_tempaddr = 2
net.ipv6.conf.default.use_tempaddr = 2
* Applying /etc/sysctl.d/10-kernel-hardening.conf ...
kernel.kptr_restrict = 1
<output_omitted>
```

12. Install the necessary key for the software to install

```
root@cp:~# sudo mkdir -p /etc/apt/keyrings
root@cp:~# curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

root@cp:~# echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

13. Install the containerd software.

```
root@cp:~# apt-get update && apt-get install containerd.io -y
root@cp:~# containerd config default | tee /etc/containerd/config.toml
root@cp:~# sed -e 's/SystemdCgroup = false/SystemdCgroup = true/g' -i /etc/containerd/config.toml
root@cp:~# systemctl restart containerd
```

```
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
<output_omitted>
```

14. Add a new repo for kubernetes. You could also download a tar file or use code from GitHub. Create the file and add an entry for the main repo for your distribution. We are using the Ubuntu 20.04 but the kubernetes-xenial repo of the software, also include the key word main. Note there are four sections to the entry.

```
root@cp:~# vim /etc/apt/sources.list.d/kubernetes.list
```

```
deb http://apt.kubernetes.io/ kubernetes-xenial main
```

15. Add a GPG key for the packages. The command spans three lines. You can omit the backslash when you type. The OK is the expected output, not part of the command.

```
root@cp:~# curl -s \
https://packages.cloud.google.com/apt/doc/apt-key.gpg \
| apt-key add -
```

```
OK
```

16. Update with the new repo declared, which will download updated repo information.

```
root@cp:~# apt-get update
```

```
<output-omitted>
```

17. Install the Kubernetes software. There are regular releases, the newest of which can be used by omitting the equal sign and version information on the command line. Historically new versions have lots of changes and a good chance of a bug or five. As a result we will hold the software at the recent but stable version we install. In a later lab we will update the cluster to a newer version.

```
root@cp:~# apt-get install -y kubeadm=1.28.1-00 kubelet=1.28.1-00 kubectl=1.28.1-00
```

```
<output-omitted>
```

```
root@cp:~# apt-mark hold kubelet kubeadm kubectl
```

```
kubelet set on hold.
kubeadm set on hold.
kubectl set on hold.
```

18. Find the IP address of the primary interface of the cp server. The example below would be the `ens4` interface and an IP of `10.128.0.3`, yours may be different. There are two ways of looking at your IP addresses.

```
root@cp:~# hostname -i
```

```
10.128.0.3
```

```
root@cp:~# ip addr show
```

```
....
2: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1460 qdisc mq state UP group default qlen 1000
   link/ether 42:01:0a:80:00:18 brd ff:ff:ff:ff:ff:ff
   inet 10.128.0.3/32 brd 10.128.0.3 scope global ens4
       valid_lft forever preferred_lft forever
   inet6 fe80::4001:aff:fe80:18/64 scope link
       valid_lft forever preferred_lft forever
....
```

19. Add an local DNS alias for our cp server. Edit the `/etc/hosts` file and add the above IP address and assign a name `k8scp`.

```
root@cp:~# vim /etc/hosts
```

```
10.128.0.3 k8scp    #<-- Add this line
127.0.0.1 localhost
....
```

20. Create a configuration file for the cluster. There are many options we could include, and they differ for **containerd**, **Docker**, and **cri-o**. Use the file included in the course tarball. After our cluster is initialized we will view other default values used. Be sure to use the node alias we added to `/etc/hosts`, not the IP so the network certificates will continue to work when we deploy a load balancer in a future lab. The file is also in the course tarball.

```
root@cp:~# vim kubeadm-config.yaml
```

YA
ML

kubeadm-config.yaml

```
1 apiVersion: kubeadm.k8s.io/v1beta3
2 kind: ClusterConfiguration
3 kubernetesVersion: 1.28.1 #<-- Use the word stable for newest version
4 controlPlaneEndpoint: "k8scp:6443" #<-- Use the alias we put in /etc/hosts not the IP
```

21. Initialize the cp. Scan through the output. Expect the output to change as the software matures. At the end are configuration directions to run as a non-root user. The token is mentioned as well. This information can be found later with the **kubeadm token list** command. The output also directs you to create a pod network to the cluster, which will be our next step. Pass the network settings **Cilium** has in its configuration file. **Please note:** the output lists several commands which following exercise steps will complete.

```
root@cp:~# kubeadm init --config=kubeadm-config.yaml --upload-certs \
| tee kubeadm-init.out #<-- Save output for future review
```

```
[init] Using Kubernetes version: v1.28.1
[preflight] Running pre-flight checks
```

<output_omitted>

You can now join any number of the control-plane node running the following command on each as root:

```
kubeadm join k8scp:6443 --token vapzqi.et2p9zbkzk29wwth \
--discovery-token-ca-cert-hash
↳ sha256:f62bf97d4fba6876e4c3ff645df3fca969c06169dee3865aab9d0bca8ec9f8cd \
--control-plane --certificate-key
↳ 911d41fcada89a18210489afaa036cd8e192b1f122ebb1b79cce1818f642fab8
```

Please note that the certificate-key gives access to cluster sensitive data, keep it secret!

As a safeguard, uploaded-certs will be deleted in two hours; If necessary, you can use "kubeadm init phase upload-certs --upload-certs" to reload certs afterward.

Then you can join any number of worker nodes by running the following on each as root:

```
kubeadm join k8scp:6443 --token vapzqi.et2p9zbkzk29wwth \
--discovery-token-ca-cert-hash
↳ sha256:f62bf97d4fba6876e4c3ff645df3fca969c06169dee3865aab9d0bca8ec9f8cd
```

22. As suggested in the directions at the end of the previous output we will allow a non-root user admin level access to the cluster. Take a quick look at the configuration file once it has been copied and the permissions fixed.

```
root@cp:~# exit
```

```
logout
```

```
student@cp:~$ mkdir -p $HOME/.kube
```

```
student@cp:~$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

```
student@cp:~$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

```
student@cp:~$ less .kube/config
```

```
apiVersion: v1
clusters:
- cluster:
<output_omitted>
```

23. Deciding which pod network to use for Container Networking Interface (CNI) should take into account the expected demands on the cluster. There can be only one pod network per cluster, although the **CNI-Genie** project is trying to change this.

The network must allow container-to-container, pod-to-pod, pod-to-service, and external-to-service communications. We will use **Cilium** as a network plugin which will allow us to use Network Policies later in the course. Currently **Cilium** does not deploy using CNI by default.

Cilium is generally installed using "cilium install" or using "helm install" commands. We have generated the cilium-cni.yaml file using the below commands for your convenience. **Note:** You don't need to execute the commands in this box, they are just for reference.

```
$ helm repo add cilium https://helm.cilium.io/
$ helm repo update
$ helm template cilium cilium/cilium --version 1.14.1 \
  --namespace kube-system > cilium.yaml
```

```
student@cp:~$ find $HOME -name cilium-cni.yaml
student@cp:~$ kubectl apply -f /home/student/LFS258/SOLUTIONS/s_03/cilium-cni.yaml
```

```
serviceaccount/cilium created
serviceaccount/cilium-operator created
secret/cilium-ca created
configmap/cilium-config created
<output_omitted>
```

24. While many objects have short names, a **kubectl** command can be a lot to type. We will enable **bash** auto-completion. Begin by adding the settings to the current shell. Then update the `$HOME/.bashrc` file to make it persistent. Ensure the `bash-completion` package is installed. If it was not installed, log out then back in for the shell completion to work.

```
student@cp:~$ sudo apt-get install bash-completion -y

<exit and log back in>

student@cp:~$ source <(kubectl completion bash)

student@cp:~$ echo "source <(kubectl completion bash)" >> $HOME/.bashrc
```

25. Test by describing the node again. Type the first three letters of the sub-command then type the **Tab** key. Auto-completion assumes the default namespace. Pass the namespace first to use auto-completion with a different namespace. By pressing **Tab** multiple times you will see a list of possible values. Continue typing until a unique name is used. First look at the current node (your node name may not start with cp), then look at pods in the `kube-system` namespace. If you see an error instead such as `-bash: _get_comp_words_by_ref: command not found` revisit the previous step, install the software, log out and back in.

```
student@cp:~$ kubectl des<Tab> n<Tab><Tab> cp<Tab>

student@cp:~$ kubectl -n kube-s<Tab> g<Tab> po<Tab>
```

26. Explore the **kubectl help** command. The output has been omitted from commands. Take a moment to review help topics.

```
student@cp:~$ kubectl help

student@cp:~$ kubectl help create
```

27. View other values we could have included in the `kubeadm-config.yaml` file when creating the cluster.

```
student@cp:~$ sudo kubeadm config print init-defaults
```

```
apiVersion: kubeadm.k8s.io/v1beta3
bootstrapTokens:
- groups:
  - system:bootstrappers:kubeadm:default-node-token
  token: abcdef.0123456789abcdef
  ttl: 24h0m0s
  usages:
  - signing
```

```
- authentication
kind: InitConfiguration
<output_omitted>
```